

CMP2204 Introduction to Computer Networks - Project Report

Platform and Development Environment

- Platform: This application was developed on Windows 10 •

Language: Python 3.9

- External Libraries:
 - Rich (for enhanced console UI)
 - pyDes (for Triple DES encryption)

Team Members and Division of Work

- Emre Kurdoğlu: Implementation of peer discovery and service announcer processes
- Ege Arda Tekin: Implementation of chat responder and chat initiator processes
- Alperen Çelik: Implementation of encryption/decryption algorithms and user interface

Project Overview

Our project implements a peer-to-peer chat application called MUD (Minimal User Protocol) as specified in the CMP2204 assignment. The application allows users on the same Local Area Network to discover each other and exchange both encrypted and unencrypted messages. The system consists of four main processes working together to provide a seamless chat experience.

Implementation Details

1. Peer Discovery Process

The peer discovery process is implemented in the `udp_listener()` function. This function creates a UDP socket that listens on port 6000 for broadcast messages from other users on the network. When a message is received, it is parsed to extract the username and IP address of the sender. If the sender is not the current user, their information is either added to the list of active users (if they are not already known) or their "last seen" timestamp is updated (if they are already known). The system also displays a notification when a new user comes online.

2. Service Announcer Process

The service announcer process is implemented in the `udp_broadcast()` function. This function periodically broadcasts the current user's username and IP address to the network using UDP on port 6000. The broadcasts occur every 8 seconds, allowing other users on the network to discover the current user's presence. The broadcast message is formatted as a JSON string containing the username and IP address.

3. Chat Responder Process

The chat responder process is implemented in the `tcp_server()` and `handle_client()` functions. The TCP server listens on port 6001 for incoming connection requests. When a connection is established, a new thread is spawned to handle the client, allowing multiple chat sessions to occur simultaneously.

The `handle_client()` function processes incoming messages, determining whether they are part of a key exchange for secure communication or regular messages. For secure communications, it:

1. Receives the peer's public key
2. Generates its own private and public keys
3. Computes the shared secret
4. Uses the shared secret to decrypt the incoming message using Triple DES encryption

For both secure and unsecure communications, messages are displayed in the console with appropriate formatting and logged to a chat history file.

4. Chat Initiator Process

The chat initiator process is implemented in the `tcp_client()` function. This function establishes a TCP connection to the specified IP address on port 6001 and sends a message. If secure communication is requested, it:

1. Generates private and public keys
2. Sends the public key to the peer
3. Receives the peer's public key
4. Computes the shared secret
5. Encrypts the message using Triple DES with the shared secret before sending

For both secure and unsecure communications, sent messages are displayed in the console with appropriate formatting and logged to a chat history file.

Encryption Implementation

For secure messaging, we implemented the Diffie-Hellman key exchange protocol followed by Triple DES encryption:

1. We use fixed parameters: $p = 19$ (prime modulus) and $g = 2$ (generator)
2. Each party generates a random private key and computes a public key: $\text{public_key} = g^{\text{private_key}} \bmod p$
3. The parties exchange public keys
4. Each party computes the shared secret: $\text{secret} = \text{peer_public_key}^{\text{private_key}} \bmod p$
5. The shared secret is padded to create a 24-byte key for Triple DES encryption

6. Messages are encrypted/decrypted using Triple DES with PKCS5 padding
7. Base64 encoding is used for transferring the encrypted binary data over the network

User Interface Enhancements

The application features several user interface improvements:

1. An ASCII art banner displays at startup for visual appeal
2. User credentials are stored for returning users
3. Messages are displayed in styled panels with clear indication of message direction and security status
4. Smooth text animations using the `slow_print()` function enhance readability
5. Color-coded messages improve usability: green for sent messages, yellow for received messages
6. Status indicators show whether users are Online or Away based on their last activity timestamp

Additional Features

New features introduced in this version include:

1. Persistent username storage - the application remembers returning users
2. Enhanced chat history viewing with timestamps and security information
3. The ability to send messages to the last contacted user with the `sendlast` command
4. Improved error handling for connection failures and malformed messages
5. Message bubbles with visual styling for better message differentiation

Challenges Faced

1. Network Configuration: One of the main challenges was ensuring that the UDP broadcast reached all users on different network configurations. The broadcast address (192.168.1.255) might need adjustment depending on the subnet mask of the network.
2. Threading and Concurrency: Managing multiple threads for the UDP broadcaster, UDP listener, TCP server, and client handling required careful implementation to avoid race conditions, particularly when updating the shared list of active users and accessing the console for output.
3. Encryption Implementation: We upgraded from simple XOR encryption to Triple DES for enhanced security. Implementing the Diffie-Hellman key exchange followed by Triple DES encryption required careful handling of key generation, padding, and binary data transmission.
4. User Interface: Creating a user-friendly console interface that could display chat messages, user statuses, and command prompts while allowing for asynchronous message reception was challenging. The Rich library helped significantly with this aspect, allowing us to create message bubbles and styled text.

Wireshark Capture Analysis

Wi-Fi'den yakalama

DOĞYA Düzenle Görünüm Git Yakala Analiz İstatistikler Telefon Kablosuz Araçlar Yardım

tcp.port == 6001

No.	Time	Source	Destination	Protocol	Length	Info
1762.	370.244710	192.168.1.3	192.168.1.4	TCP	66	6001 → 64783 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1762.	370.250802	192.168.1.3	192.168.1.4	TCP	64	6001 → 64783 [PSH, ACK] Seq=1 Ack=11 Win=65280 Len=10
1762.	370.275492	192.168.1.3	192.168.1.4	TCP	54	6001 → 64783 [ACK] Seq=11 Ack=69 Win=65280 Len=0
1762.	370.278296	192.168.1.3	192.168.1.4	TCP	54	6001 → 64783 [FIN, ACK] Seq=11 Ack=69 Win=65280 Len=0
1762.	370.241087	192.168.1.4	192.168.1.3	TCP	66	64783 → 6001 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1762.	370.244709	192.168.1.4	192.168.1.3	TCP	54	64783 → 6001 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1762.	370.245003	192.168.1.4	192.168.1.3	TCP	64	64783 → 6001 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=10
1762.	370.269098	192.168.1.4	192.168.1.3	TCP	111	64783 → 6001 [PSH, ACK] Seq=11 Ack=11 Win=65280 Len=57
1762.	370.273314	192.168.1.4	192.168.1.3	TCP	54	64783 → 6001 [FIN, ACK] Seq=68 Ack=11 Win=65280 Len=0
1762.	370.278349	192.168.1.4	192.168.1.3	TCP	54	64783 → 6001 [ACK] Seq=69 Ack=12 Win=65280 Len=0
2093.	439.958048	192.168.1.4	192.168.1.3	TCP	66	64810 → 6001 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2093.	439.961669	192.168.1.3	192.168.1.4	TCP	66	6001 → 64810 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2093.	439.961727	192.168.1.4	192.168.1.3	TCP	54	64810 → 6001 [ACK] Seq=1 Ack=1 Win=65280 Len=0
2093.	439.961837	192.168.1.4	192.168.1.3	TCP	97	64810 → 6001 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=43
2093.	439.963619	192.168.1.4	192.168.1.3	TCP	54	64810 → 6001 [FIN, ACK] Seq=44 Ack=1 Win=65280 Len=0
2093.	439.966141	192.168.1.3	192.168.1.4	TCP	54	6001 → 64810 [ACK] Seq=1 Ack=45 Win=65280 Len=0
2093.	439.970359	192.168.1.3	192.168.1.4	TCP	54	6001 → 64810 [FIN, ACK] Seq=1 Ack=45 Win=65280 Len=0
2093.	439.970406	192.168.1.4	192.168.1.3	TCP	54	64810 → 6001 [ACK] Seq=45 Ack=2 Win=65280 Len=0

Ethernet II, Src: LiteonTechno_23:bf:97 (c0:35:32:23:bf:97), Dst: Intel_03:0b:ec (20:1e:88:03:0b:ec)

Internet Protocol Version 4, Src: 192.168.1.4, Dst: 192.168.1.3

Transmission Control Protocol, Src Port: 64810, Dst Port: 6001, Seq: 1, Ack: 1, Len: 43

Source Port: 64810

Destination Port: 6001

[Stream index: 87]

[Stream Packet Number: 4]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 43]

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 323565113

[Next Sequence Number: 44 (relative sequence number)]

Acknowledgment Number: 1 (relative ack number)

Acknowledgment number (raw): 3026797447

0101 = Header Length: 20 bytes (5)

Flags: 0x018 (PSH, ACK)

Window: 255

[Calculated window size: 65280]

Window size scaling factor: 256]

Checksum: 0xf215 [unverified]

[Checksum Status: Unverified]

0000 20 1e 88 03 0b ec c0 35 32 23 bf 97 08 00 45 00 5 2# E

0010 00 53 5d f1 40 00 80 06 19 5c c0 a8 01 04 c0 a8 S] @ S

0020 01 03 fd 2a 17 71 c0 dc 2d c9 b4 69 43 87 50 18 * q i C P

0030 00 ff f2 15 00 00 7b 22 75 6e 65 6e 63 72 79 70 { " unencrypt

0040 74 65 64 20 6d 65 73 73 61 67 65 22 3a 20 22 41 ted mess age": "A

0050 6c 70 65 72 65 6e 20 45 67 65 20 45 6d 72 65 22 }peren E ge Emre"

0060 7d

wireshark_Wi-FiHBBAG2.pcapng

Paketler: 288751 · Displayed: 18 (0.0%)

Profil: Default

17:59

6.05.2025

Wi-Fi'den yakalama

DOĞYA Düzenle Görünüm Git Yakala Analiz İstatistikler Telefon Kablosuz Araçlar Yardım

tcp.port == 6001

No.	Time	Source	Destination	Protocol	Length	Info
1762.	370.244710	192.168.1.3	192.168.1.4	TCP	66	6001 → 64783 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1762.	370.250802	192.168.1.3	192.168.1.4	TCP	64	6001 → 64783 [PSH, ACK] Seq=1 Ack=11 Win=65280 Len=10
1762.	370.275492	192.168.1.3	192.168.1.4	TCP	54	6001 → 64783 [ACK] Seq=11 Ack=69 Win=65280 Len=0
1762.	370.278296	192.168.1.3	192.168.1.4	TCP	54	6001 → 64783 [FIN, ACK] Seq=11 Ack=69 Win=65280 Len=0
1762.	370.241087	192.168.1.4	192.168.1.3	TCP	66	64783 → 6001 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1762.	370.244789	192.168.1.4	192.168.1.3	TCP	54	64783 → 6001 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1762.	370.245003	192.168.1.4	192.168.1.3	TCP	64	64783 → 6001 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=10
1762.	370.269098	192.168.1.4	192.168.1.3	TCP	111	64783 → 6001 [PSH, ACK] Seq=11 Ack=11 Win=65280 Len=57
1762.	370.273314	192.168.1.4	192.168.1.3	TCP	54	64783 → 6001 [FIN, ACK] Seq=68 Ack=11 Win=65280 Len=0
1762.	370.278349	192.168.1.4	192.168.1.3	TCP	54	64783 → 6001 [ACK] Seq=69 Ack=12 Win=65280 Len=0
2093.	439.958048	192.168.1.4	192.168.1.3	TCP	66	64810 → 6001 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2093.	439.961669	192.168.1.3	192.168.1.4	TCP	66	6001 → 64810 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
2093.	439.961727	192.168.1.4	192.168.1.3	TCP	54	64810 → 6001 [ACK] Seq=1 Ack=1 Win=65280 Len=0
2093.	439.961837	192.168.1.4	192.168.1.3	TCP	97	64810 → 6001 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=43
2093.	439.963619	192.168.1.4	192.168.1.3	TCP	54	64810 → 6001 [FIN, ACK] Seq=44 Ack=1 Win=65280 Len=0
2093.	439.966141	192.168.1.3	192.168.1.4	TCP	54	6001 → 64810 [ACK] Seq=1 Ack=45 Win=65280 Len=0
2093.	439.970359	192.168.1.3	192.168.1.4	TCP	54	6001 → 64810 [FIN, ACK] Seq=1 Ack=45 Win=65280 Len=0
2093.	439.970406	192.168.1.4	192.168.1.3	TCP	54	64810 → 6001 [ACK] Seq=45 Ack=2 Win=65280 Len=0

Ethernet II, Src: LiteonTechno_23:bf:97 (c0:35:32:23:bf:97), Dst: Intel_03:0b:ec (20:1e:88:03:0b:ec)

Internet Protocol Version 4, Src: 192.168.1.4, Dst: 192.168.1.3

Transmission Control Protocol, Src Port: 64783, Dst Port: 6001, Seq: 11, Ack: 11, Len: 57

Source Port: 64783

Destination Port: 6001

[Stream index: 78]

[Stream Packet Number: 6]

[Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 57]

Sequence Number: 11 (relative sequence number)

Sequence Number (raw): 49489837

[Next Sequence Number: 68 (relative sequence number)]

Acknowledgment Number: 11 (relative ack number)

Acknowledgment number (raw): 3216916489

0101 = Header Length: 20 bytes (5)

Flags: 0x018 (PSH, ACK)

Window: 255

[Calculated window size: 65280]

Window size scaling factor: 256]

Checksum: 0x0c51 [unverified]

[Checksum Status: Unverified]

0000 20 1e 88 03 0b ec c0 35 32 23 bf 97 08 00 45 00 5 2# E

0010 00 61 5d ec 40 00 80 06 19 53 c0 a8 01 04 c0 a8 a] @ S

0020 01 03 fd 0f 17 71 02 f3 27 ad bf be 40 09 50 18 * q i C P

0030 00 ff 0c 51 00 00 7b 22 65 6e 63 72 79 70 74 65 ... Q . { " encrypte

0040 64 20 6d 65 73 73 61 67 65 22 3a 20 22 30 71 7a d messag e": "0qz

0050 62 40 42 68 46 69 4f 2b 6c 37 77 4a 57 70 61 75 bHbHf10+ 17wJkxau

0060 4f 69 2b 78 55 53 6d 34 68 4b 6a 4f 71 22 7d 01+XuSm4 hKj0q*")

wireshark_Wi-FiHBBAG2.pcapng

Paketler: 287715 · Displayed: 18 (0.0%)

Profil: Default

17:58

6.05.2025

Conclusion

Our implementation successfully meets all the requirements specified in the project assignment:

1. It implements the four required processes: Peer Discovery, Service Announcer, Chat Responder, and Chat Initiator
2. It successfully detects all available users in the LAN
3. It provides end-user interactions as specified, including display of user status and options for secure/unsecure messaging
4. It enables successful chat with any available user in the LAN
5. It successfully encrypts/decrypts peer-to-peer conversations using Triple DES encryption
6. It generates a chat log with timestamps and message content
7. It provides enhanced user experience with visual styling and improved usability features

The application provides a practical demonstration of network programming concepts, including socket programming, protocols (TCP/UDP), threading, cryptography, and user interface design. We have extended the basic requirements with additional features like persistent username storage, enhanced message bubbles, and improved chat history functionality to create a more complete and user-friendly application.